LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING

CVE-2026-39833 Invoking key constraints not enforced in golang.org/x/crypto/ssh/agent

Vulnerability Report

Classification	TLP:CLEAR
Published	2026-06-11
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Report

Published: 2026-06-11 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary

CVE-2026-39833 is a critical authorization-bypass vulnerability affecting the in-memory SSH agent keyring implementation in `golang.org/x/crypto/ssh/agent`. Public Go security reporting and the linked Go issue state that `NewKeyring()` silently accepted the `ConfirmBeforeUse` key constraint but never enforced it. As a result, an application could believe a loaded key required user confirmation before signing while the keyring actually signed requests without any prompt. The public remediation changes this behavior so unsupported constraints now return an error rather than failing open.

This is not a generic internet-facing SSH daemon flaw and it is not described as a memory-corruption issue. The operational risk is concentrated in software that embeds the Go `ssh/agent` package, constructs an in-memory keyring with `NewKeyring()`, and relies on confirmation-gated key use as a security boundary. In those environments, an attacker who reaches the signing path may be able to obtain SSH signatures or authentication operations that should have required an explicit user-presence check, undermining bastion access controls, privileged Git operations, or internal remote-access workflows.

The reviewed public source set does not indicate in-the-wild exploitation, proof-of-concept weaponization, or CISA KEV inclusion as of 2026-06-11. The highest-confidence defensive action is still immediate because this flaw weakens a trust control rather than merely creating noisy instability: organizations should upgrade `golang.org/x/crypto` to `v0.52.0` or later, identify applications that instantiate `agent.NewKeyring()`, and review whether any internal services or developer tooling assumed `ConfirmBeforeUse` was enforceable prior to the update.

Executive takeaway	Assessment
Business impact	Silent bypass of an SSH key confirmation control can allow unauthorized signing or authentication activity in applications that trusted `ConfirmBeforeUse` to enforce user approval.
Most exposed environments	Internal tools, remote-access platforms, developer workflows, or automation services that embed `golang.org/x/crypto/ssh/agent` and use the in-memory keyring rather than an external OpenSSH agent.
Public exploitation status	No reviewed public source confirmed in-the-wild exploitation or KEV inclusion as of 2026-06-11.
Immediate priority	Upgrade to `golang.org/x/crypto v0.52.0+`, inventory code paths using `agent.NewKeyring()`, and review privileged SSH-signing workflows that assumed confirmation prompts were enforced.

2. Intelligence Requirement

This review addresses the following intelligence requirement: what does the current public record say about `CVE-2026-39833`, which software patterns are actually exposed, how should defenders think about exploitation likelihood, and what concrete actions should engineering, AppSec, and detection teams take next?

Question	Assessment
What is the vulnerability?	A fail-open constraint-enforcement flaw in the in-memory keyring returned by `golang.org/x/crypto/ssh/agent.NewKeyring()`: `ConfirmBeforeUse` could be accepted but never enforced.
What is affected?	Applications and services using `golang.org/x/crypto/ssh/agent`, specifically the in-memory keyring implementation, with public remediation references converging on `golang.org/x/crypto` versions earlier than `v0.52.0`.
What is the practical impact?	Keys that operators expected to require an interactive confirmation

	step could instead be used silently for signing or authentication.
Does this affect every SSH deployment?	No. The highest-confidence exposure is code that embeds the vulnerable Go package and relies on its in-process keyring semantics; the reviewed public record does not claim a blanket flaw in external SSH agents.
Is exploitation publicly confirmed?	No. The reviewed public source set did not show public exploitation, a released exploit chain, or KEV listing as of 2026-06-11.
What matters most to defenders?	Dependency upgrade, code-path discovery, and review of applications where SSH-agent confirmation was being treated as a meaningful authorization gate.

3. Source Review and Confidence

Source	Contribution	Confidence
Microsoft Security Update Guide entry	Confirms the intake event, title, and publication chronology that drove this review.	Medium
Go issue `#79436`	Most direct public technical description of the flaw: `NewKeyring()` accepted `ConfirmBeforeUse` but never enforced it, and now errors on unsupported constraints.	High
Go vulnerability entry `GO-2026-5005` on Go Packages	Canonical Go ecosystem advisory identifier, publication date, affected package, and remediation pointer.	High
OSV entry for `GO-2026-5005`	Corroborates the Go vulnerability record and distribution into broader dependency-scanning ecosystems.	High
CVE Record / CVE references	Corroborates the CVE identifier and ties the issue to the Go change lists and announcement references.	High
Go security announcement (`golang-announce`)	Supports chronology and broader Go ecosystem notification.	Medium
CISA vulnerability summary bulletin	Independent cataloging of the issue and additional public-sector visibility.	Medium
Downstream package-remediation references (for example dependency bump issues and package scanners)	Corroborate that defenders and downstream projects are converging on `v0.52.0` as the fixed package version.	Medium

Confidence is high on the core mechanics of the vulnerability because the public Go issue describes the failure mode directly: unsupported confirmation constraints were silently accepted by the in-memory keyring and not enforced, enabling signing without the expected confirmation prompt. Confidence is also high that the correct immediate remediation is upgrading away from vulnerable `golang.org/x/crypto` releases because the Go vulnerability record and multiple downstream package-maintenance references converge on `v0.52.0` as the fixed version.

Confidence is lower on any claim about broad real-world exploitation because the public reporting reviewed for this task is sparse and primarily developer-oriented. The available sources support strong statements about flawed authorization semantics

and exposed coding patterns, but they do not support overconfident claims about exploit kits, named threat actors, or widespread opportunistic abuse.

4.1 Core Vulnerability Metadata

Field	Value
CVE	`CVE-2026-39833`
Go advisory ID	`GO-2026-5005`
Vulnerability name	`Invoking key constraints not enforced in golang.org/x/crypto/ssh/agent`
Affected package	`golang.org/x/crypto`
Affected component	`golang.org/x/crypto/ssh/agent`
Vulnerable behavior	`agent.NewKeyring()` accepted `ConfirmBeforeUse` but did not enforce it
Security consequence	Keys could sign without the confirmation prompt that callers expected to act as a user-approval gate
Fixed behavior	`NewKeyring()` now returns an error when unsupported constraints are requested
Fixed version	`v0.52.0`
Public publication date	`2026-05-22` on the Go vulnerability page
MSRC intake event reviewed	`2026-06-11`
Public exploitation at review time	Not confirmed in reviewed sources
KEV status at review time	Not listed in reviewed public results

4.2 What Is Actually Exposed

Exposure pattern	Why it matters	Confidence
Applications that call `agent.NewKeyring()` and load keys with `ConfirmBeforeUse` expectations	The vulnerable keyring is the component explicitly described as failing open.	High
Internal developer tooling that embeds an SSH agent abstraction in-process	These tools often depend on confirmation prompts to reduce silent privileged key use.	Medium
Remote-access or orchestration platforms using Go SSH-agent code for brokered authentication	A compromised or malicious caller may obtain a signature without the operator interaction the platform thought it required.	Medium
CI/CD or automation wrappers reusing agent semantics as an approval guardrail	This can turn a human-in-the-loop control into a silent allow path if the service trusted the library behavior.	Medium
Generic OpenSSH agent deployments with no embedded vulnerable Go keyring	The reviewed public sources do not support a blanket claim of exposure here.	High

4.3 Realistic Attack Scenarios

Scenario	Description	Defensive implication
Embedded admin tool bypass	A privileged internal tool loads an SSH key into an in-memory Go keyring with `ConfirmBeforeUse`, assuming operators must approve each sign request. An attacker who reaches that tool's sign path can use the key silently.	Review every sign-capable service or tool that treated confirmation prompts as a hard authorization boundary.
Developer-workstation workflow abuse	A local or adjacent attacker abuses a developer tool, plugin, or remote session that relies on the vulnerable in-process keyring for Git, bastion, or infrastructure authentication.	Focus response on developer and platform engineering tooling, not just servers.
Automation trust-boundary failure	A service uses confirmation-constrained keys to separate routine actions from privileged SSH-backed actions, but the application silently signs anyway because the library failed open.	Patch plus design review is required; remediation is not only a version bump but also a verification of assumed control semantics.

4.4 Analyst Findings

Finding	Defensive implication
This is a security-control-bypass issue, not a reliability bug.	Prioritize it wherever SSH confirmation was used as an explicit policy control.
The vulnerable surface is narrower than a network daemon RCE, but potentially high-value.	Exposure discovery must be code- and dependency-centric rather than perimeter-centric.
Public reporting points to a fail-open design outcome: unsupported constraints were accepted instead of rejected.	Security reviews should check for similar “accepted but ignored” control semantics in adjacent authentication code.
No reviewed public source supplied stable exploit IOCs or malware artifacts.	Detection should focus on asset inventory, code search, build-pipeline scanning, and unusual SSH-signing behavior rather than signature-only IOC matching.

5. Threat Context

CVE-2026-39833 is best understood as an application-level authorization bypass in the software supply chain around Go-based SSH tooling. The flaw does not hand an attacker arbitrary code execution on its own. Instead, it weakens a control that some developers or platform teams may have assumed provided user-presence assurance before a private key could be used. That distinction matters because a vulnerability like this can remain invisible during routine testing: software appears to support a security option, but the option is silently non-functional.

The most consequential environments are those where SSH keys unlock privileged infrastructure access, source-control administration, bastion workflows, deployment pipelines, or production automation. In such cases, confirmation gating may be the only control preventing a remote workflow, plugin, or service component from silently invoking a sensitive key. If the application design assumed that the Go keyring enforced that gate, then the real blast radius includes unauthorized SSH authentication, unintended signing operations, and reduced operator visibility into key use.

No reviewed public source tied the CVE to a named threat actor, campaign, or observed exploitation set as of 2026-06-11. Even so, the issue is attractive to capable adversaries because it undermines trust around already-loaded credentials rather than stealing them outright. Organizations should therefore think about this bug in the same risk family as other “policy bypass

around sensitive credentials" defects: the vulnerability may not create exposure everywhere, but where it does apply, it can neutralize an otherwise meaningful safeguard.

MITRE ATT&CK mapping	Relevance	Confidence
`T1550.004` Use Alternate Authentication Material: SSH Keys	Plausible if an attacker abuses the vulnerable keyring to perform SSH authentication or signing with a key that operators expected to be confirmation-gated.	Medium
`T1078` Valid Accounts	Relevant where the silently usable key enables access that is operationally equivalent to misuse of a legitimate account or credentialed trust path.	Low
Additional ATT&CK techniques	Not assigned with high confidence because the public record does not describe a full intrusion chain or observed post-exploitation behavior.	High

6. Detection and Hunting

This vulnerability is best hunted as a dependency-and-code-exposure problem first, and a runtime-behavior problem second. Most organizations will not get useful value from searching for a single IOC because the public record does not provide exploit artifacts, hashes, domains, or concrete intrusion telemetry. The strongest detection posture is to identify where vulnerable package versions are present, find code paths that instantiate `agent.NewKeyring()`, and then prioritize those applications for upgrade, design review, and any available audit of SSH-signing behavior.

6.1 Detection Logic Summary

Signal	Telemetry source	Analytic goal
`golang.org/x/crypto` below `v0.52.0` in `go.mod`, `go.sum`, SBOMs, or dependency scanners	SCA platforms, SBOM repositories, CI dependency checks, artifact manifests	Identify vulnerable builds and packages
Source code importing `golang.org/x/crypto/ssh/agent` and calling `agent.NewKeyring()`	Code search, Semgrep, ripgrep, SAST	Find the exposed implementation pattern described in the advisory
Code paths that set `ConfirmBeforeUse` or rely on confirmation semantics for sensitive keys	Source review, unit tests, architecture docs, code search	Distinguish mere package presence from materially risky control reliance
Services brokering SSH signing/authentication through embedded Go agent logic	Service inventory, platform architecture review, application telemetry	Prioritize high-impact workflows for emergency patching and validation
Unexpected SSH authentication or sign operations from tools that were believed to require explicit approval	Service logs, access broker logs, Git/bastion audit trails, application logs	Detect possible historical silent use in high-value environments

Analyst caveat: many applications will include `golang.org/x/crypto` without using the affected in-memory keyring path. Treat simple package presence as a triage lead, not final proof of exploitable business risk.

6.2 Example Detection / Hunt Content

Sigma-style secure-code review stub aligned to the supporting detection artifact:

```
title: Review Go In-Memory SSH Agent Keyring Usage for CVE-2026-39833
id: cve-2026-39833-go-newkeyring-review
status: experimental
description: Review projects that instantiate golang.org/x/crypto/ssh/agent.NewKeyring() and may rely
on ConfirmBeforeUse semantics.
logsource:
  product: source-code
  category: static_analysis
detection:
  selection_import:
    Import|contains: 'golang.org/x/crypto/ssh/agent'
  selection_call:
    Code|contains: 'NewKeyring('
  condition: selection_import and selection_call
level: medium
```

Kusto example for dependency-led exposure hunting:

```
let vulnerableProjects = externaldata(Project:string, Dependency:string, Version:string)
[
  @"https://example.invalid/replace-with-your-sbom-or-dependency-export.csv"
]
with(format="csv", ignoreFirstRecord=true);
vulnerableProjects
| where Dependency =~ "golang.org/x/crypto"
| where Version !startswith "0.52." and Version !startswith "0.53." and Version !startswith "0.54." and
Version !startswith "0.55."
| project Project, Dependency, Version
```

Repository hunt for vulnerable code patterns and confirmation semantics:

```
rg -n --glob='*.go' 'NewKeyring\(|ConfirmBeforeUse|AddedKey\{|ssh/agent' .
```

Dependency-version verification in Go build environments:

```
go list -m all | grep 'golang.org/x/crypto'
```

6.3 Hunting Guidance

Hunt question	Why it matters
Which repos, binaries, or containers still embed `golang.org/x/crypto` below `v0.52.0`?	This establishes the immediate remediation set.
Which of those projects actually call `agent.NewKeyring()`?	This narrows the vulnerable population from generic package consumers to directly exposed code paths.
Which applications rely on user confirmation before SSH signing or authentication?	These are the places where the bypass most directly defeats an intended control.
Do internal bastion, Git, or remote-access workflows show silent sign or authentication operations that operators expected to require approval?	This helps distinguish theoretical exposure from meaningful historical misuse.
Did engineering teams document confirmation prompts as a compensating control in architecture or threat models?	Such documentation indicates elevated business dependence on the bypassed security property.

7. Recommendations

Priority	Owner	Action
Critical	Application Security / Engineering	Upgrade `golang.org/x/crypto` to `v0.52.0` or later in every affected repository, rebuild dependent binaries, and redeploy artifacts.

Critical	Platform Engineering	Inventory software that imports `golang.org/x/crypto/ssh/agent` and explicitly identify uses of `agent.NewKeyring()`.
High	Security Architecture / Service Owners	Review whether any application treated `ConfirmBeforeUse` as an authorization boundary and document compensating controls if human approval was expected for privileged key use.
High	Detection Engineering / DevSecOps	Add CI/SCA gates that flag `golang.org/x/crypto` versions below `v0.52.0` and code patterns invoking `agent.NewKeyring()` in sensitive projects.
High	IAM / Infrastructure Security	Review bastion, Git, and internal remote-access workflows where embedded Go SSH-agent logic could have enabled silent use of privileged keys.
Medium	QA / Engineering	Add regression tests that fail closed when unsupported key constraints are requested, so future control regressions do not silently pass.
Medium	Threat Intelligence	Monitor future Go advisories, OSV updates, and public exploit reporting for any change in exploitation status or additional technical detail.

Additional analyst guidance:

- Treat this as both a patching issue and a control-validation issue. A version bump alone is not enough if teams still assume security properties they have not tested end-to-end.
- Prioritize software that mediates administrator SSH access, production deployment, or source-control signing because the value of a silently usable key is highest in those workflows.
- Where feasible, validate patched behavior in a lab by attempting to request unsupported constraints and confirming the application now fails closed.

8. References

- 1. Microsoft Security Update Guide - CVE-2026-39833: https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-39833
- 2. Go issue `#79436` - `x/crypto/ssh/agent: lifetime without confirm constraint`: https://github.com/golang/go/issues/79436
- 3. Go vulnerability report `GO-2026-5005`: https://pkg.go.dev/vuln/GO-2026-5005
- 4. OSV entry for `GO-2026-5005`: https://osv.dev/vulnerability/GO-2026-5005
- 5. CVE Record: https://www.cve.org/CVERecord?id=CVE-2026-39833
- 6. Go security announcement thread: https://groups.google.com/g/golang-announce/c/a082jnz-LvI
- 7. CISA vulnerability summary bulletin for the week of 2026-05-18: https://www.cisa.gov/news-events/bulletins/sb26-145